

DBMS Term Project Proposal

Topic: Cost-Based Query Optimization Engine with Heuristic Pruning

Team Name: DBMSTeam_404

Group Members

Roll Number	Name
23CS10054	Popuri Anirudh
23CS10013	Chintada Yesheeth Sree Narayana
23CS10078	Vemuri Sai Lokesh
23CS10052	Payidiparthi Sri Surya Naga Hadwik
23CS10017	Dasari Veera Venkata Abhinav Teja

Abstract

This project develops a standalone cost-based query optimizer designed to mathematically evaluate and restructure database operations. The core implementation focuses on building an algorithmic engine that will take simple, unoptimized execution plans as input and evaluate alternate execution paths. The system will then output optimized execution plans, along with associated cost measure.

The system architecture consists of three primary components:

- **Execution Plan Ingestion and Representation:** A module that ingests unoptimized logical trees (via JSON) supporting core SPJ (Select-Project-Join) operations alongside GROUP BY/HAVING clauses, building an appropriate representation of query plan nodes in memory.
- **Cost-Based Optimizer Engine:** A mathematical component that will use (faked) table statistics to choose optimal plans. This engine will properly cost different plan nodes by utilizing formulas that take CPU, memory, disk requirements into account.
- **Search Space Pruning Engine:** A dynamic programming component that explores various join orders and execution paths. It will integrate a mechanism for limiting the search effort of the optimizer to prevent combinatorial explosion during complex queries.

Stretch Goal: If the core algorithmic components are completed ahead of schedule, the system will be extended to parse raw SQL DML commands and extract real table statistics from a live database, bypassing the need for direct AST and faked statistics inputs.

Motivation

Database query performance remains a critical challenge in data-intensive applications. As query complexity grows, specifically with multi-table joins, the number of possible execution paths explodes exponentially. This project addresses this gap by creating a custom query optimization layer

that leverages mathematical cost models and dynamic programming to make cost-aware optimization decisions. By isolating the optimizer from the storage engine, we can focus entirely on the algorithmic efficiency of execution path generation and heuristic search pruning.

Implementation Plan

Week 1: Plan Representation and Ingestion

- Design and implement an appropriate representation of query plan nodes using C++ or Python object-oriented structures.
- Develop the ingestion parser to take simple, unoptimized execution plans as input using JSON formats.

Week 2: Statistics and Cost Modeling

- Develop a mock catalog system to use (faked) table statistics to choose optimal plans.
- Implement the mathematical costing functions to properly cost different plan nodes.
- Ensure the cost models strictly take CPU, memory, disk requirements into account.

Week 3: Core Optimization Engine

- Build the primary search algorithm to evaluate different execution paths and join orders, specifically mapping logical joins to competing physical operators (e.g., Hash Join, Nested Loop Join, Merge Join).
- Develop the output formatter to successfully output optimized execution plans, along with associated cost measure.

Week 4: Search Pruning & Stretch Goals

- Integrate a mechanism for limiting the search effort of the optimizer using dynamic programming memoization to explore Left-Deep and/or Bushy join trees, discarding high-cost branches early.
- **Extension:** Begin implementing a raw SQL string parser and live PostgreSQL statistics connection if the core engine is fully functional.

Week 5: Integration, Testing, and Evaluation

- Integrate all components into a cohesive, standalone executable.
- Develop a test suite utilizing complex, multi-table joins to evaluate the performance of the pruning mechanism.
- Document optimization techniques, cost models, and their impact in a final report, aiming for a report length should be around 10 pages.

Technical Requirements

- **Programming language:** C++
- Standard JSON libraries for parsing unoptimized execution tree structures.
- **(Stretch Goal)** PostgreSQL database for real statistics extraction and SQL string parsing integration.

Additional Scope and Goals (will be implemented if time is available)

- **Bitmask Dynamic Programming:** Implement bitmask-based state transitions for $O(2^N)$ optimal join enumeration, minimizing memory overhead and accelerating the optimizer's core search engine.
- **A* Search Graph Traversal:** Model the execution plan search space as a Directed Acyclic Graph (DAG) and apply the A* algorithm with custom mathematical heuristics to aggressively prune sub-optimal paths.
- **Multi-Objective Pareto Optimization:** Upgrade the scalar cost model to evaluate a mathematical cost vector, outputting a Pareto frontier of execution plans that mathematically balance CPU, memory, and disk I/O trade-offs.
- **Hardware-Aware Non-Linear Costing:** Introduce complex step-functions to the mock statistics to dynamically simulate expensive hardware events, such as $O(N \log N)$ disk-spilling when intermediate tables exceed simulated RAM capacity.

Expected Outcomes

- A functional cost-based query planner capable of mathematically evaluating and optimizing unoptimized execution trees.
- An interface for entering unoptimized plans and receiving the optimized output alongside calculated CPU, memory, and disk cost metrics.
- Insights into the effects of search pruning strategies (e.g., dynamic programming vs. exhaustive search) on the optimization engine's speed and efficiency.